RETAILPULSE

AI-Powered Customer Analytics & Demand Forecasting
Transforming Retail Intelligence with Data-Driven Insights



RetailPulse Command Center — deployed dashboard view

ZIDIO INTERNSHIP PROJECT REPORT

Developed By: **Govind Suthar**
Project Duration: **09 October 2026 – 03 November 2026**

Live Dashboard: Open RetailPulse
GitHub: Govind99603 / RetailPulse

Project Overview

RetailPulse is a retail analytics dashboard built to make transaction data easier to understand. The project takes the Online Retail II dataset and presents analysis of revenue, customer behavior, products, seasonality, demand, and inventory-related signals through a multi-page Streamlit application.

Vision & Objectives

• Explore sales performance and revenue movement across the available transaction period.

• Understand customer activity using Recency, Frequency, and Monetary (RFM) analysis.

• Review product performance and daily demand summaries.

• Explore demand forecasting outputs and inventory reorder-related indicators.

• Make the analysis accessible through an interactive dashboard and public deployment.

Target Users / Use Cases

• Retail analysts and business analysts exploring sales patterns.

• Store or e-commerce teams reviewing customer and product insights.

• Learners and project reviewers examining an end-to-end analytics workflow.

Business Value

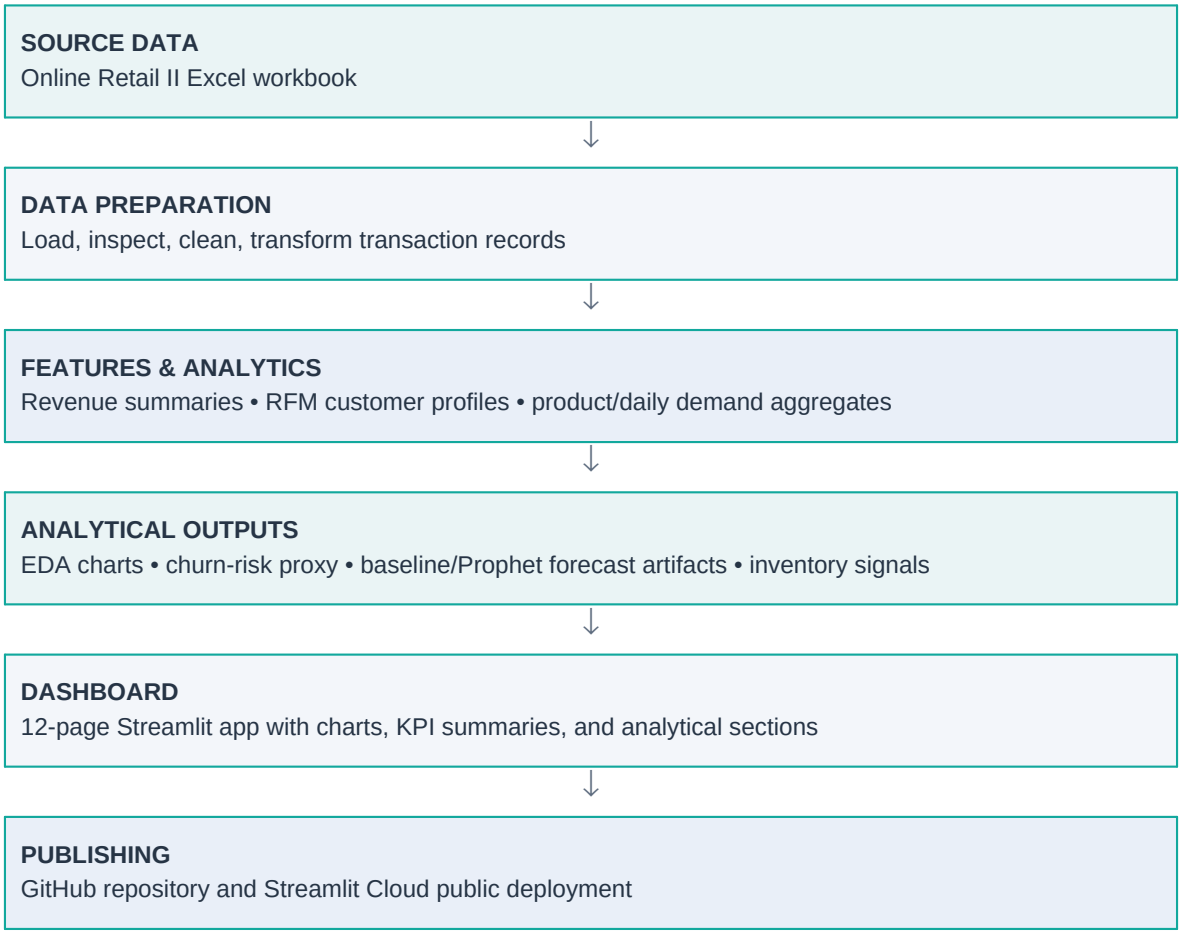
The dashboard brings several analytical views into one place. It can support exploratory questions about sales, customer segments, product demand, and inventory planning. The outputs are decision-support signals and should be considered alongside business context and data quality.

Project Scope

The completed deliverable includes prepared data and analysis artifacts, a 12-page Streamlit dashboard, repository hosting, and a live application. Forecasting is presented as baseline/directional analysis; production-grade monitoring and automated retraining are not represented as fully implemented.

Architecture Overview

RetailPulse follows a practical data-to-dashboard workflow. The architecture below reflects the project's implemented flow, with ML operations components treated as supporting or future extensions unless evidenced as active.



Architecture Notes

• Data analysis and feature preparation were performed with Python-based tools.

• Streamlit presents the analysis as navigable pages rather than a single static report.

• MLflow experiment logging and PSI helper functionality are present in the notebook workflow where dependencies/configuration allow; comprehensive live monitoring is not claimed.

Execution Timeline

The project was carried out from 09 October 2026 to 03 November 2026. The timeline below organizes the completed work into stages across the project period.

Stage / Dates	Work completed
Stage 1 · 09–13 Oct 2026	Requirement review, source workbook loading, dataset inspection, data quality checks, and initial cleaning.
Stage 2 · 14–19 Oct 2026	Exploratory analysis including headline metrics, quantity/price distributions, monthly revenue, country revenue, and product summaries.
Stage 3 · 20–25 Oct 2026	Date-based feature engineering, customer RFM calculations and segments, churn-proxy target preparation, model pipeline training/evaluation.
Stage 4 · 26–29 Oct 2026	Daily demand aggregation, baseline forecast evaluation, Prophet/LSTM workflow exploration, and product inventory/reorder calculations.
Stage 5 · 30 Oct–03 Nov 2026	Streamlit dashboard page organization, deployment and repository finalization, chart/report assets preparation, and final project documentation.

Project milestones

- Prepared analysis artifacts from the Online Retail II dataset.
- Built a dashboard organized into 12 analytical pages.
- Published the application on Streamlit Cloud and maintained the source on GitHub.
- Completed the project report and notebook documentation.

The dates are a summarized project timeline, not a claim that each task was performed only within its listed stage.

Technical Highlights

Modeling & Analytical Techniques

• Customer analysis using RFM measures and segment labels.

• Churn-risk proxy modeling using a prepared customer activity target and a logistic-regression pipeline.

• Daily demand aggregation with a naive baseline forecast evaluated on a held-out time period.

• Prophet and LSTM forecast workflows represented in the notebook where the required libraries were available.

• Product-level average daily demand and reorder-related calculations using defined assumptions.

Feature Engineering

• Recency, frequency, and monetary value at customer level.

• Calendar attributes such as year, month, weekday, hour, and weekend indicator.

• Customer activity windows for a churn-proxy target.

• Daily and product-level demand aggregates.

Challenges Faced

• Handling transaction data quality concerns such as duplicates, missing identifiers, and unusual quantities/prices.

• Interpreting customer metrics and forecast outputs without overstating their reliability.

• Organizing multiple analytical outputs into a coherent dashboard experience.

• Managing deployment and report assets across notebook, repository, and hosted app.

How I addressed them

• Used a staged cleaning and validation workflow before analysis.

• Separated exploratory summaries and modeling outputs by analytical purpose.

• Created focused dashboard pages to make insights easier to navigate.

• Kept forecast and inventory outputs described as baseline signals where full validation was not established.

Deployment & Operations

Deployment Platform

• Streamlit Cloud hosts the public RetailPulse dashboard.

• GitHub stores the source code and project files.

• The application is structured as a multi-page Streamlit dashboard.

Application URL

https://govind99603-retailpulse-app-yqxrxq.streamlit.app/

Repository URL

https://github.com/Govind99603/RetailPulse

Operational Notes

The deployed application is intended for interactive exploration of the project's prepared analytical data. The app's page interactions and all filter/data-state combinations should be regarded as subject to further testing. Docker, continuous integration, automated refresh, and monitoring integrations are not stated as fully operational deliverables in this report.



Figure: RetailPulse Command Center hosted dashboard view

Personal Reflection

Working on RetailPulse gave me practical experience in taking a retail dataset through analysis and into a dashboard that can be accessed online. I worked through data checks and cleaning, explored business patterns, prepared customer and product summaries, and connected those outputs to a Streamlit interface.

Key Learnings

• How to inspect and prepare transaction-level data for analytics.

• How to use RFM measures to summarize customer behavior.

• How to communicate analytical results with charts and dashboard pages.

• How to structure a Python analytics project and publish it using GitHub and Streamlit Cloud.

• Why model evaluation, assumptions, and data quality matter when presenting forecast or risk-related outputs.

Practices Applied

• Modular organization of analysis and dashboard topics.

• Version control and source-code hosting with GitHub.

• Clear visual presentation of metrics and analytical results.

• Documentation of project workflow, implementation scope, and limitations.

Future Roadmap Ideas

• Extend time-series backtesting and publish model comparison metrics.

• Review anomalous SKU demand and refine reorder assumptions.

• Add stronger automated data quality and app interaction tests.

• Explore scheduled data refresh and monitoring if the application is developed for production use.

The project improved my confidence in building a practical analytics deliverable from raw data through to a deployed dashboard.

Key Features

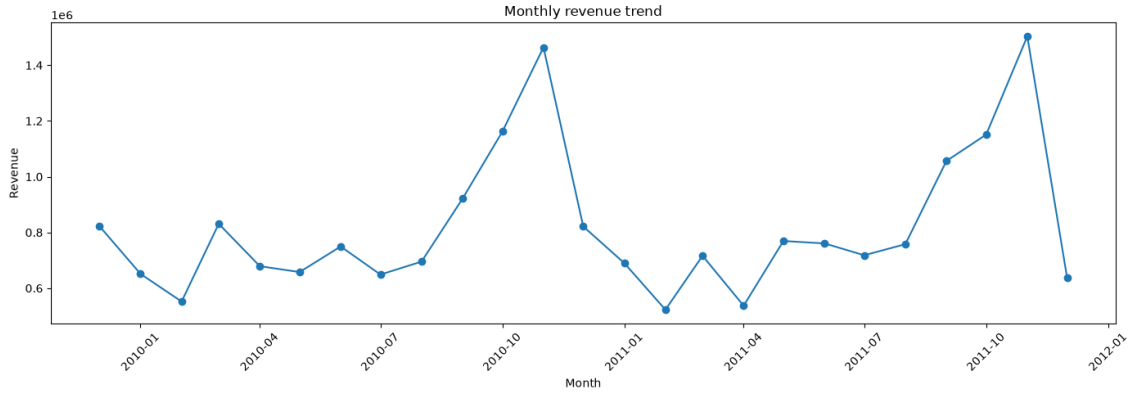
ID	Feature	Description
F1	Revenue Analysis	Monthly and overall revenue trend exploration.
F2	Customer Segments	Customer profiling and RFM segment distribution.
F3	Product Intelligence	Product-level performance and demand summaries.
F4	Geo & Market View	Country/market comparison analysis.
F5	Seasonality Lab	Calendar-based sales pattern exploration.
F6	Basket Behavior	Invoice/basket-related analysis.
F7	Inventory Control	Demand and reorder-related signals.
F8	Forecast Lab	Baseline and forecast visualizations.
F9	Analytics Workbench	Additional analytical exploration.

Technology Stack

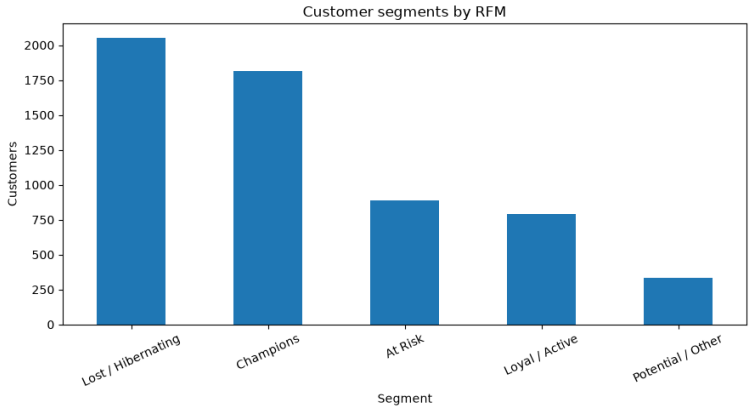
Category	Technology	Use in the project
Programming	Python	Main language for analysis and application logic.
Data Analysis	Pandas, NumPy	Data preparation, transformation, aggregation.
Dashboard	Streamlit	Multi-page web dashboard.
Visualization	Plotly / Matplotlib (where used)	Charts and visual exploration.
ML tools	Scikit-learn	Churn-proxy pipeline and ML workflow components.
Forecasting	Prophet, TensorFlow/Keras (optional in notebook)	Forecast workflow experimentation.
Experiment tracking	MLflow (conditional notebook workflow)	Experiment logging when available/configured.
Versioning & hosting	Git, GitHub, Streamlit Cloud	Repository and app publication.

Visuals

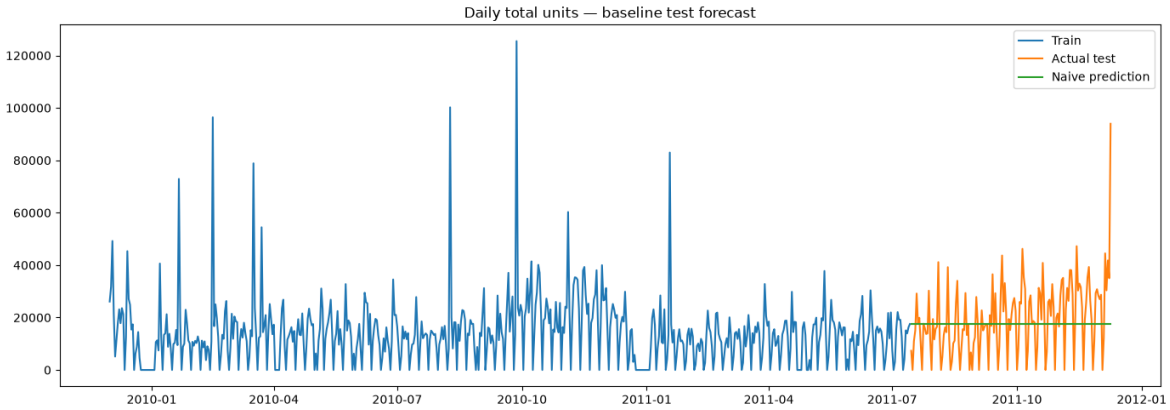
The following figures are outputs from the project notebook and analysis artifacts.



Monthly Revenue Trend

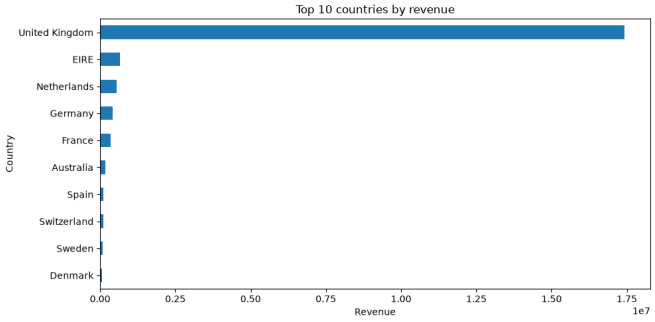


Customer RFM Segments

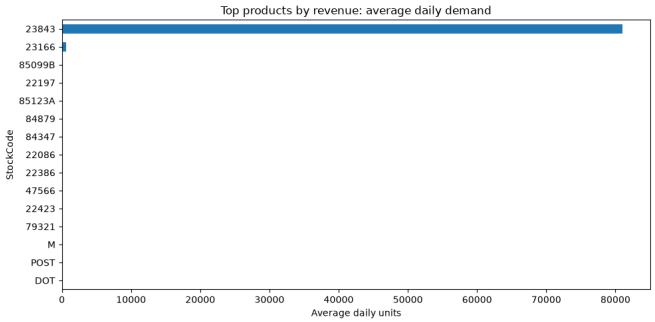


Daily Demand Baseline Forecast

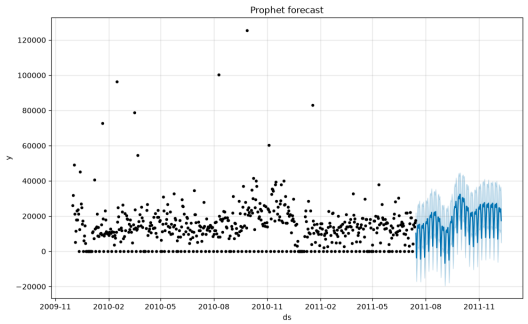
Project Visuals, Output & Conclusion



Revenue by Top Countries



Average Daily Demand by Product



Prophet Forecast Visualization

OUTPUT

The final output is a publicly accessible 12-page Streamlit retail analytics dashboard, accompanied by the project repository, prepared data/analysis artifacts, notebook, and project report.

CONCLUSION

RetailPulse demonstrates a practical end-to-end retail analytics workflow using the Online Retail II dataset. The project combines data preparation, exploratory analysis, customer RFM segmentation, product and revenue insights, demand-related analysis, and a deployed Streamlit dashboard. It provided hands-on learning in data processing, analytics, visualization, and deployment. The work is a portfolio-scale deliverable, with forecast validation, anomaly review, automated testing, and production monitoring identified as reasonable areas for further development.

Designed & Built by Govind Suthar